

Cybersecurity & Blockchain

KGSP Enrichment [Week 3 & 4 – 2026]





Mudassar Aslam

(FHEA, PhD Cybersecurity)

Associate Professor of Cybersecurity & Program Director,
School of Engineering, Computing and Mathematics,
Oxford Brookes University,
Oxford, United Kingdom.
maslam@brookes.ac.uk

www.mudassir.info



Day 10 – Smart Contracts



Introduction

- Smart contracts are **digital contracts** stored on a blockchain that are **automatically executed** when **predetermined terms** and conditions are met.
 - automate the execution of an agreement
 - No intermediary's involvement which saves time
 - The contract inherits the trust of blockchain
- They can automate a workflow, triggering the next action when predetermined conditions are met
- Smart contracts work by the following simple “**if/when...then...**” statements that are written into code on a blockchain. A network of computers executes the actions when predetermined conditions are met and verified.
- **Examples:** releasing funds to the appropriate parties, registering a vehicle, sending notifications or issuing a ticket



Benefits of Smart Contracts



▪ Speed, efficiency and accuracy

- Once a condition is met, the contract is executed immediately. Because smart contracts are digital and automated, there's no paperwork to process and no time spent reconciling errors that often result from manually completing documents.



▪ Trust and transparency

- Because there's no third party involved, and because encrypted records of transactions are shared across participants, there's no need to question whether information has been altered for personal benefit.



▪ Security

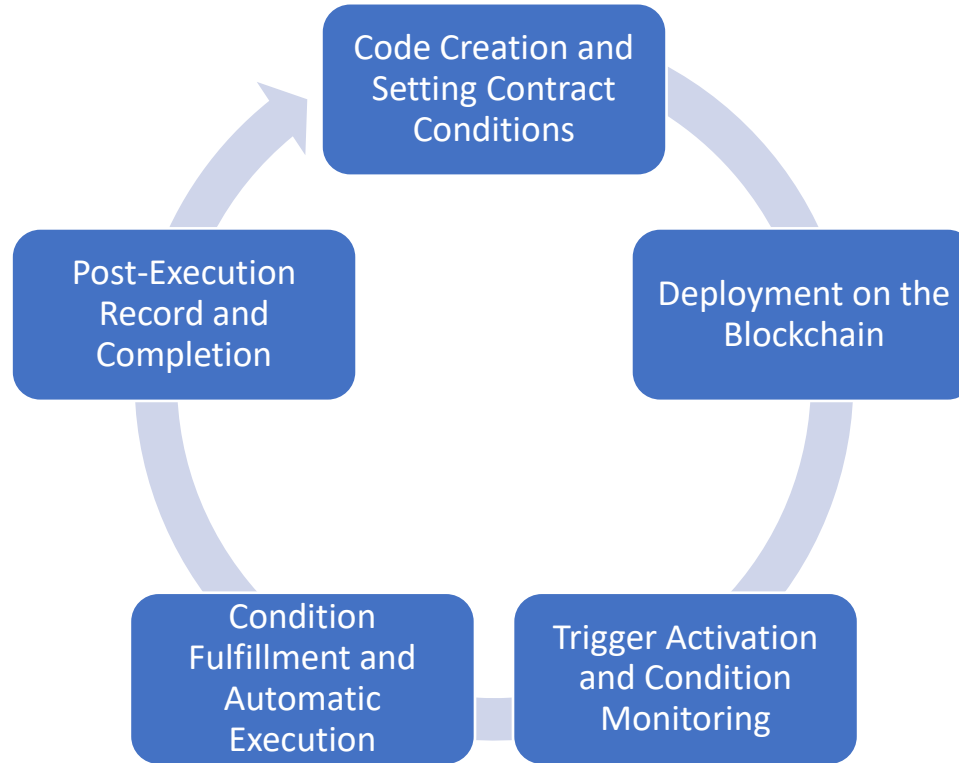
- Blockchain transaction records are encrypted, which makes them hard to hack. Moreover, because each record is connected to the previous and subsequent records on a distributed ledger, hackers have to alter the entire chain to change a single record.



▪ Savings

- Smart contracts remove the need for intermediaries to handle transactions and, by extension, their associated time delays and fees.

Setting up a Smart Contract



1. Code Creation and Setting Contract Conditions

- The foundation of a smart contract lies in the program code defining its terms.
- The code outlines
 - the start conditions,
 - triggers, and
 - actions to be executed.
- For instance, a smart contract for selling a product may state, "Ship the item when the buyer pays the specified amount."
- Such code is primarily written in programming languages adopted by blockchains like Ethereum, such as [Solidity](#).

2. Deployment on the Blockchain

- Once the code is created, it is uploaded to the blockchain network and deployed.
- This makes the smart contract part of a decentralized database shared across the network.
- This step ensures the transparency of the code, allowing all stakeholders to verify the contract terms.
- The risk of tampering is extremely low, guaranteeing reliability.

3. Trigger Activation and Condition Monitoring



- Contract execution requires specific conditions, known as **triggers**.
- These conditions are automatically monitored within the blockchain.
- Examples
 - completion of payment
 - confirmation of receipt of goods.
- **Oracle**: In some cases, external data is integrated through a mechanism called "oracle," allowing external environmental data to be used as conditions.



4. Condition Fulfillment and Automatic Execution



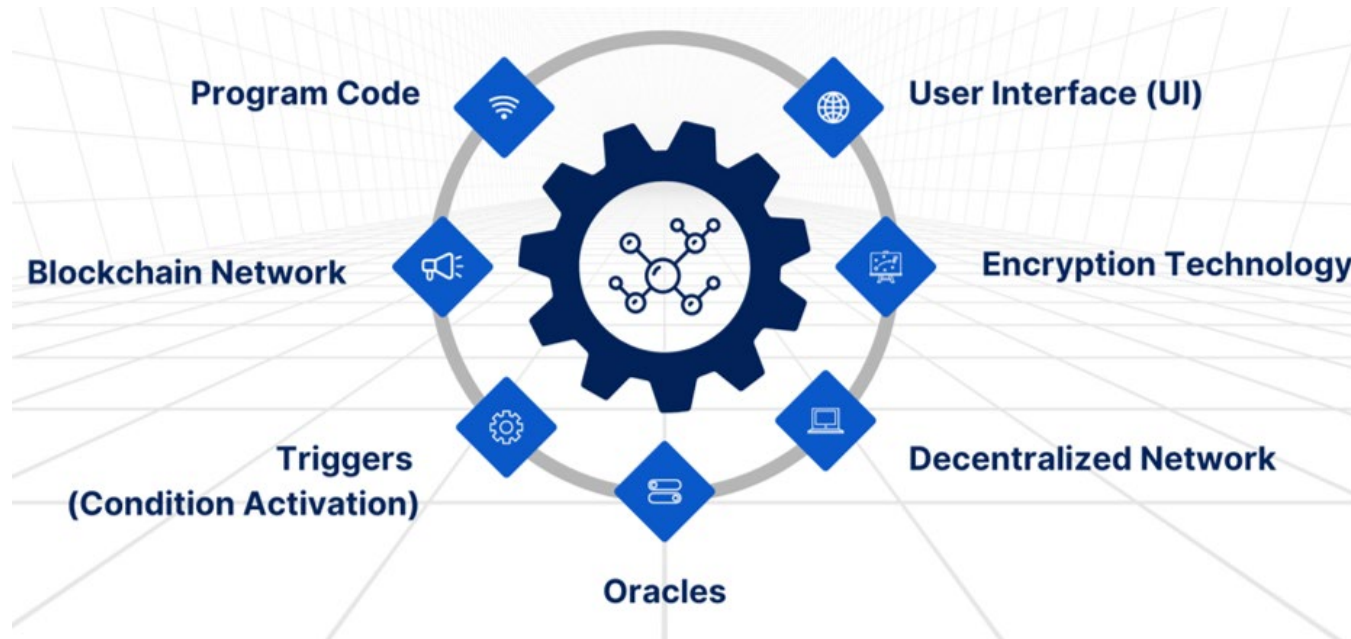
- When the conditions are satisfied, the smart contract executes automatically according to the program.
- Example:
 - once payment is confirmed, the digital ownership of a product is transferred automatically.
- Eliminates human intervention
- Reduces the risk of errors or delays
- Execution details are recorded on the blockchain, ensuring transparency and allowing all stakeholders to verify the outcomes.



5. Post-Execution Record and Completion

- After a smart contract is executed, the result is permanently recorded on the blockchain.
 - Resistant to tampering
 - Can be easily verified later
 - Reliable source of evidence in case of disputes.
- The mechanism of smart contracts significantly improves efficiency, reliability, and transparency compared to traditional contract methods.

Components of Smart Contracts



Challenges of Smart Contracts

- Dependency on External Data and Oracle Issues
 - Reliability and security of oracles can be a concern, e.g. Malicious or erroneous data from an oracle could lead to incorrect contract execution.
 - To mitigate this risk, it is essential to [use highly reliable oracles](#) or gather data from multiple sources.
- The Immutability Dilemma
 - Once deployed, smart contracts are immutable, meaning they cannot be easily changed.
 - Bugs or errors in the code can lead to significant problems.
- Legal and Regulatory Uncertainty
 - How smart contracts fit into [existing laws and regulations](#) remains unclear.
 - For contracts spanning [multiple jurisdictions](#), interpretation and enforcement can become complicated.
 - Replacing human decision-making entirely with smart contracts raises [questions about accountability and legal remedies](#) in case of disputes.



Practical

First Smart Contract Using Remix IDE



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

